Our first node with ROS2

Olivier Aycard
Professor
Grenoble INP - PHELMA
GIPSA Lab
https://olivier-aycard.github.io/
olivier.aycard@grenoble-inp.fr





Outline

1. Our first node in ROS;
2. Compile and run nodes.

Our first node

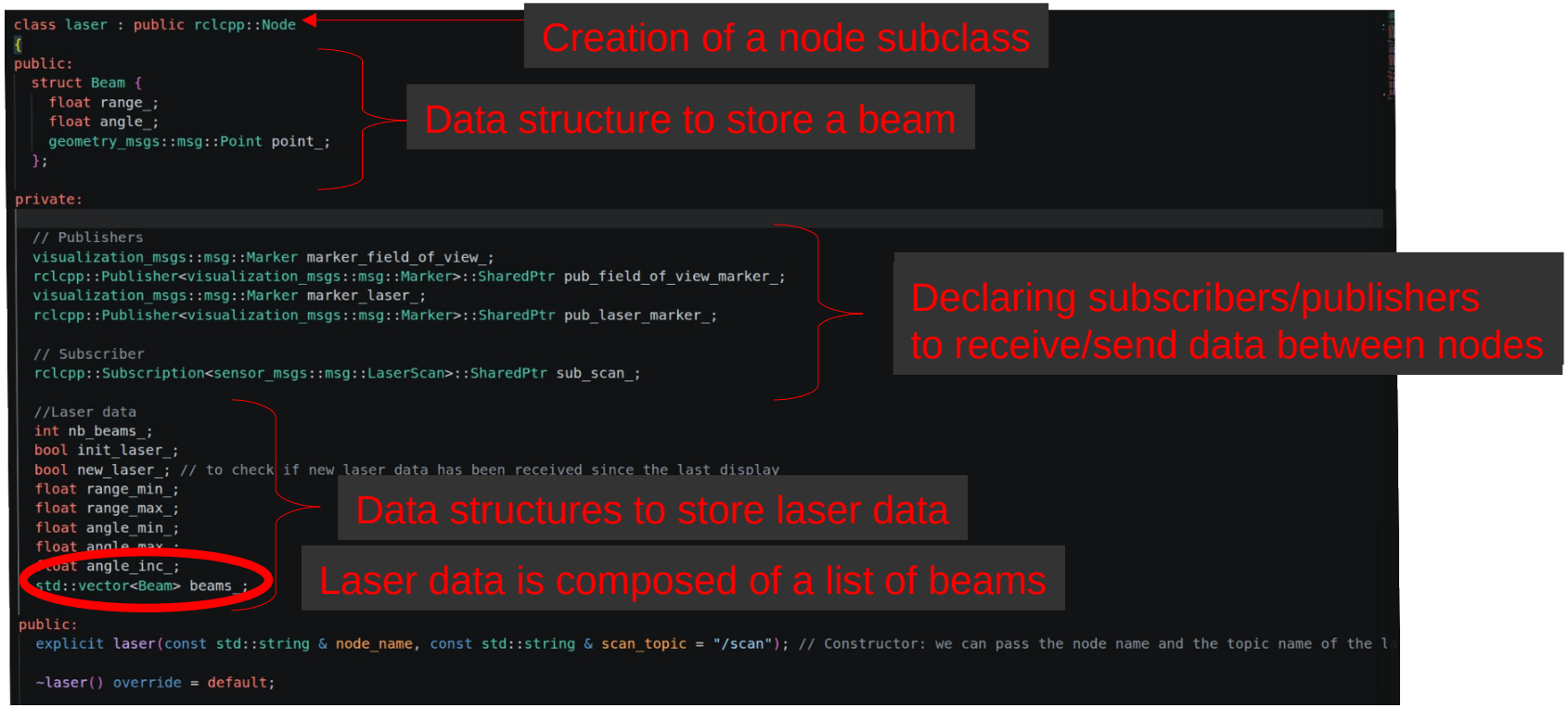
In order to demonstrate how a ROS node is implemented, let's look at a simple example which does the following :

- Interfaces with a sensor :
  - receives laser data from the LIDAR scanner
  - stores it in some data structures
- Does something with the sensor data:
  - reads the range values
  - displays them as text on the terminal
- Sends some data from this node to other nodes :
  - sends graphical display points so that we can display them on the RVIZ graphical interface

Our first node : a class to store laser data (1/5)

Let's see how a simple node can be implemented !

- In vscode, open `ros2_ws/src/follow_me/inc/laser.hpp`




```
class laser : public rclcpp::Node
{
public:
  struct Beam {
    float range_;
    float angle_;
    geometry_msgs::msg::Point point_;
  };

private:

  // Publishers
  visualization_msgs::msg::Marker marker_field_of_view_;
  rclcpp::Publisher<visualization_msgs::msg::Marker>::SharedPtr pub_field_of_view_marker_;
  visualization_msgs::msg::Marker marker_laser_;
  rclcpp::Publisher<visualization_msgs::msg::Marker>::SharedPtr pub_laser_marker_;

  // Subscriber
  rclcpp::Subscription<sensor_msgs::msg::LaserScan>::SharedPtr sub_scan_;

  //Laser data
  int nb_beams_;
  bool init_laser_;
  bool new_laser_; // to check if new laser data has been received since the last display
  float range_min_;
  float range_max_;
  float angle_min_;
  float angle_max_;
  float angle_inc_;
  std::vector<Beam> beams_;

public:
  explicit laser(const std::string & node_name, const std::string & scan_topic = "/scan"); // Constructor: we can pass the node name and the topic name of the l

  ~laser() override = default;
```

Our first node : a class to store laser data (2/5)

- Open now laser.cpp to see the implementation of methods of the laser class to process laser data

- Our first node will subscribe to the topic called « scan » with the subscriber sub_scan;
- « scan » is the topic on which messages are published by the laser, containing the laser data;
- Each time a new message is published on the topic called « scan », our first node will receive and store this message with the method « receive_scan »
- Look at the constructor of laser class

```
//subscribe to the laser scan topic ie, "/scan" by default. The callback function receive_scan() will be called each time a new scan is received.
sub_scan_ = create_subscription<sensor_msgs::msg::LaserScan>(scan_topic, rclcpp::SensorDataQoS(),
  [this](sensor_msgs::msg::LaserScan::ConstSharedPtr scan) { receive_scan(scan); });
```

Our first node : a class to store laser data (3/5)

- Each time laser data is received, the method « receive_scan » automatically processes the message

The type of messages published by the laser

```
void laser::receive_scan(sensor_msgs::msg::LaserScan::ConstSharedPtr scan)
{
  if (scan->ranges.empty() || !std::isfinite(scan->angle_min) ||
      !std::isfinite(scan->angle_increment) || scan->angle_increment <= 0.f) {
    set_new_laser(false);
    return;
  }

  // store the important data of the laser scan in the class variables
  set_init_laser(true);
  set_new_laser(true);

  set_range_min(scan->range_min);
  set_range_max(scan->range_max);
  set_angle_min(scan->angle_min);
  set_angle_max(scan->angle_max);
  set_angle_inc(scan->angle_increment);
  set_nb_beams(static_cast<int>(scan->ranges.size()));

  //loop to read and store each beam of the laser
  marker_laser_.header = scan->header;
  init();
  float beam_angle = get_angle_min();
  for (int loop = 0; loop < get_nb_beams(); ++loop, beam_angle += get_angle_inc()) {
    const float raw_range = scan->ranges[loop];
    const bool valid = std::isfinite(raw_range) && raw_range > get_range_min() && raw_range < get_range_max();
    const float range = valid ? raw_range : get_range_max();

    // transform the beam coordinates to the world frame
    geometry_msgs::msg::Point hit;
    hit.x = range * std::cos(beam_angle);
    hit.y = range * std::sin(beam_angle);
    hit.z = 0.0;
    add(range, beam_angle, hit);
  }
}//receive_scan()
```

Our first node : a class to store laser data (4/5)

- Our first node will publish to the topics called « laser_marker » and « field_of_view_marker;
- They are topics on which messages are published by our node so that other ROS nodes can receive them;
- The type of the message is `visualization_msgs::msg_marker`, which represents visual marker points with colours ;
- These graphical markers will be used by rviz2 to have a graphical display of the laser data and the field of view of the laser
- Look again at the constructor of laser class

```
// publisher to display the field of view of the laser in RViz
pub_field_of_view_marker_ = this->create_publisher<visualization_msgs::msg::Marker>("field_of_view_marker", 1);

// publisher to display the laser data in RViz
pub_laser_marker_ = create_publisher<visualization_msgs::msg::Marker>("laser_marker", 1);
```

Our first node : a class to store laser data (5/5)

- We have as well a method to display the laser data in textual and graphical mode

```
void laser::display_laser(bool graphical)
{
  if (!get_init_laser()) {
    return;
  }

  // textual display
  std::ostringstream output;
  output << std::fixed << std::setprecision(3)
         << "Laser scan: " << get_nb_beams() << " beams"
         << ", range [" << get_range_min() << ", " << get_range_max() << "] m"
         << ", angles [" << get_angle_min() << ", " << get_angle_max() << "] rad\n"
         << "index | angle (rad) | distance (m) | x (m) | y (m)\n";

  // loop to display each beam of the laser
  for (int i = 0; i < get_nb_beams(); ++i) {
    const auto & beam = get_beam_at(static_cast<std::size_t>(i));
    output << i << " | " << beam.angle_ << " rad | " << beam.range_ << " m | "
           << beam.point_.x << " m | " << beam.point_.y << " m | "
           << '\n';
  }
```

Display general info about laser

Our first node

- Now, we have to implement a node (ie, an executable) that will use the laser class and perform some processes
- Open display_laser_node.cpp
- It contains a main which is our main program

Our first node

- Our first node is an infinite loop that will run at 10 hz

1. It will check if a new message from the laser has been published (`spin_some`);
2. If a new message has been received on the scan topic, it will call the method `receive_scan` to collect the data of the laser;
3. The method « `update` » will process the data of the laser

```
int main(int argc, char ** argv)
{
  rclcpp::init(argc, argv);
  {
    auto scan = std::make_shared<laser>("laser_node");

    // The executor runs the subscription callbacks of the registered nodes.
    rclcpp::executors::SingleThreadedExecutor executor;
    // Register scan so the executor can call receive_scan() when laser data arrives.
    executor.add_node(scan->get_node_base_interface());

    rclcpp::Rate rate(10);  // 10 iterations per second.

    while (rclcpp::ok()) {
      executor.spin_some();     // Run pending callbacks before processing the data.
      scan->update();           // Display the new measurements.
      rate.sleep();             // Wait for the next iteration.
    }
  }
  rclcpp::shutdown();
  return 0;
}
```

Our first node

- The method « update » will check if new message of the laser has arrived with the methode `get_new_laser();`
- It will loop over the beams, display the field of view and laser data in the terminal, and publish a marker corresponding to the hits;

```
// Display measurements once for each new scan.
void laser::update()
{
  if (!get_init_laser()) { // check if the laser has been initialized with valid data
    RCLCPP_WARN_THROTTLE(get_logger(), *get_clock(), 2000,
      "Waiting for laser data: run a ros bag or connect to a robot.");
    return;
  }
  if (!get_new_laser()) { // check if new laser data has been received since the last display
    return;
  }

  display_field_of_view(); //display the field of view of the laser in RViz
  display_laser(); //display the laser data in RViz and print the values of each beam in the terminal

  // Mark the scan as processed and wait for the next one.
  update_state_variables();
}
```

Outline

1. Our first node in ROS;
2. Compile and run nodes.

Compile and run nodes

Now we've seen what a ROS node looks like, how do we compile and run them ?

- To compile the node: goto a tab*
- `cd ~/ros2_ws`
- `colcon build`

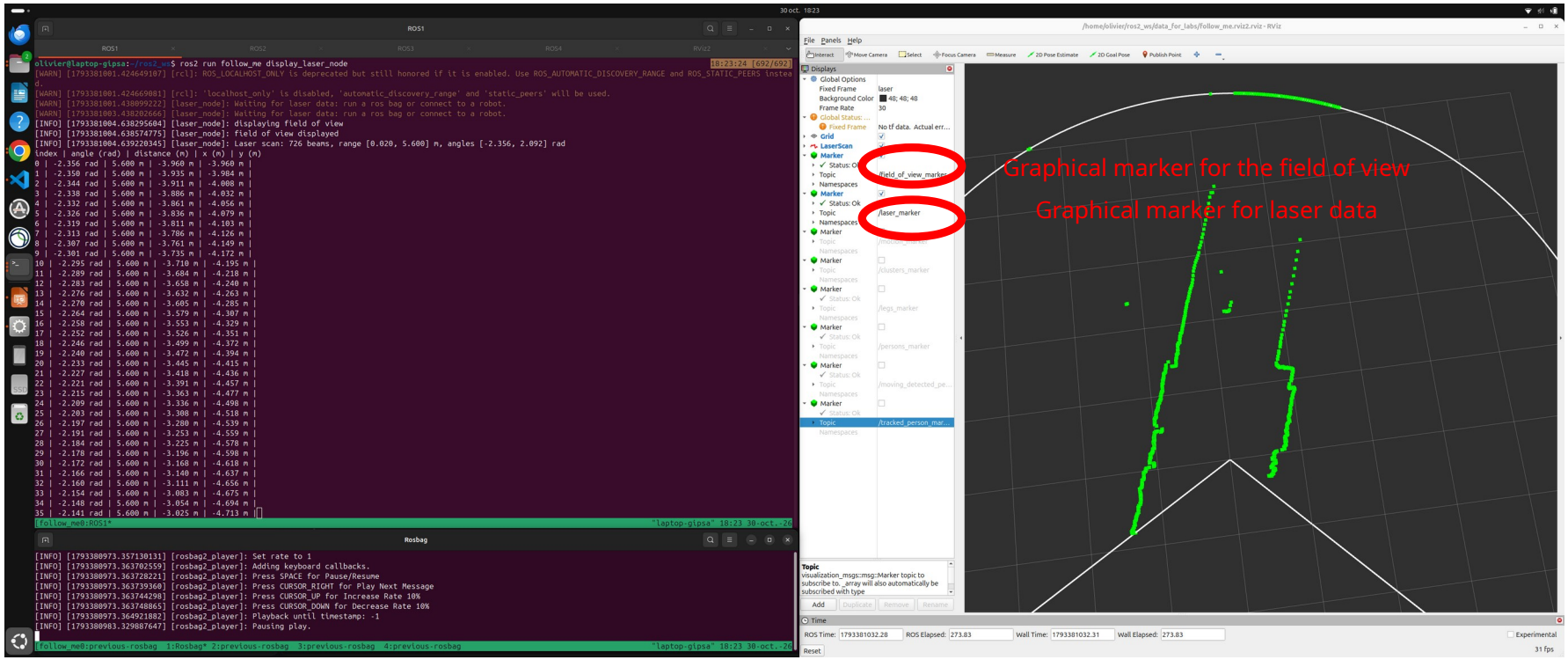
- To run the node in a tab :
- `cd ~/ros2_ws`
- `ros2 run follow_me display_laser_node`

***note:** the terminal in which you compile using `colcon build` should NOT be one where you have previously executed `source install/local_setup.bash`.*
*We recommend you always keep the **same terminal** open for compiling, and use it **ONLY** for compiling.*

Compile and run nodes

- Our node is now running, but we need to pass laser data to it as input, and we want to visualise the graphical markers it publishes as output.
-
- To run a rosbag : go to the bottom terminal :
- `ros2 bag play <data_file>.bag2`
- `Replace <data_file>.bag2 with the name of the file to play,`
- `for instance : detection01.bag2`

Results


Graphical marker for the field of view
Graphical marker for laser data

General reminders and information :

- A folder in ~/ros2_ws/src is called a package;
- A package contains some source files;
- These source files will be compiled to create nodes;
- To compile: `colcon build` from ~/ros2_ws
  For instance, in the package tutorial_ros, there is one source file that will generate one node.

- To run a node:
  `ros2 run <package_name> <node_name>` from ~/ros2_ws, after having done the `source` command once in that terminal.
  For instance, we used the command
  `ros2 run tutorial_ros laser_graphical_display_node`
  to run the node *laser_graphical_display_node* which is part of the package *tutorial_ros.*

Before moving on :

- Make sure you understand how the laser data is **stored** :
  - Which function is used to receive the data ?
  - When is it called ?
  - In which variables does it store the positions of hits ?

- Make sure you understand how the laser data is **used** :
  - On Rviz, you see that there is a blue marker for each hit of the laser. Which line of code uses the laser point coordinates to create the corresponding marker point ?
  - Are the points passed as cartesian (x,y) or polar (r, theta) points ?
  - If you wanted to create marker points only for the laser hits which have an .x coordinate less than 1m, how would you need to change the code ?